

Django Nesneleri Serileştirmek ¶

Django'nun serileştirme çerçevesi, Django modellerini başka biçimlere "dönüştürmek" için bir mekanizma sunar.



Ayrıca btk.

Tablolardaki verileri serileştirilmiş bir biçime aktarmak istiyorsanız, 'dumpdata' yönetim komutunu kullanabilirsiniz.

Verilerin Sıralanması ¶

En üst düzeyde, verileri şu şekilde serileştirebilirsiniz:

```
from django.core import serializers

data = serializers.serialize("json", SomeModel.objects.all())
```

Serialize fonksiyonunun argümanları; verinin hangi biçimde serileştirileceği (bkz. Serileştirme formatları) ve serileştirilecek olan **QuerySet**'tir. (Aslında ikinci argüman, Django model örnekleri üreten herhangi bir "iterator" olabilir; ancak bu neredeyse her zaman bir QuerySet olacaktır.)

django.core.serializers.get_serializer(format) ¶

Ayrıca Serileyici Öğeleri direkt kullanabilirsiniz.

```
JSONSerializer = serializers.get_serializer("json")
json_serializer = JSONSerializer()
json_serializer.serialize(queryset)
data = json_serializer.getvalue()
```

Bu yöntem verileri doğrudan dosya benzeri bir nesneye (**HttpResponse** dahil) serileştirmek istediğinizde kullanışlıdır:

```
with open("file.json", "w") as out:
    json_serializer.serialize(SomeModel.objects.all(), stream=out)
```



Ayrıca btk.

Bilinmeyen bir biçimle get_serializer() işlevi çağrıldığında, django.core.serializers.SerializerDoesNotExist istisnası ortaya çıkar.

Alanların Alt Kümesi ¶

Yalnızca belirli bir alt kümedeki alanların serileştirilmesini istiyorsanız, serileştiriciye bir **fields**(alan,sınıf) argümanı belirtebilirsiniz:

```
from django.core import serializers

data = serializers.serialize("json", SomeModel.objects.all(), fields=["name", "size"])
```

Bu örnekte, her modelin yalnızca **ismi** ve **boyutu**, öznitelikleri serileştirilecektir. Birincil anahtar, sonuç çıktısında her zaman **pk** ögesi olarak serileştirilir; **fields**(alan,sınıf) bölümünde asla görünmez.



Ayrıca btk.

Kullandığınız modele bağlı olarak, yalnızca alanlarının bir kısmını serileştiren bir modeli ters serileştiremeyebilirsiniz. Serileştirilmiş bir nesne, modelin gerektirdiği tüm alanları belirtmiyorsa, ters serileştirici ters serileştirilmiş örnekleri kaydedemez.

Kalıtılabilir Modeller ¶

Eğer **soyut bir temel** sınıf kullanılarak tanımlanmış bir modeliniz varsa, bu modeli serileştirmek için özel bir işlem yapmanız gerekmez. Seri hale getirmek istediğiniz nesne (veya nesneler) üzerinde seri hale getirme fonksiyonunu çağırın; çıktı, seri hale getirilmiş nesnenin tam bir temsiliyi içerecektir. Ancak, **çoklu tablo mirasımı** kullanan bir modeliniz varsa, modelin tüm temel sınıflarını da serileştirmeniz gerekir. Bunun nedeni, yalnızca model üzerinde yerel olarak tanımlanan alanların serileştirilecek olmasıdır. Örneğin, aşağıdaki modelleri ele alalım:

```
class Place(models.Model):
    name = models.CharField(max_length=50)

class Restaurant(Place):
    serves_hot_dogs = models.BooleanField(default=False)
```

Yalnızca Restaurant modelini serileştirirseniz:

```
data = serializers.serialize("json", Restaurant.objects.all())
```

Seri hale getirilmiş çıktıda yer alan alanlar yalnızca “**serves_hot_dogs**” özniteliğini içerecektir. Ana sınıfın “**İsim**”özniteliği göz ardı edilecektir.

“**Restaurant**” nesnelerinizi tam olarak seri hale getirmek için, “**Place**” modellerini de seri hale getirmeniz gerekecektir

```
all_objects = [*Restaurant.objects.all(), *Place.objects.all()]
data = serializers.serialize("json", all_objects)
```

Veri serileştirilmesinin kaldırılması ¶

Verilerin serileştirilmesinden sonra kaldırılması, serileştirme işlemine çok benzerdir:

```
for obj in serializers.deserialize("json", data):
    do_something_with(obj)
```

Gördüğünüz gibi, “**deserialize**(Veri Serileştirmesinin kaldırılması)” işlevi “**serialize**(veri Serileştirmesi)”ile aynı format argümanını (bir dize veya veri akışı) alır ve bir yineleyici döndürür.

Ancak burada işler biraz karmaşıklaşıyor.” **Deserialize**(Veri Serileştirmesinin kaldırılması)” yineleyicisi tarafından döndürülen nesneler, sıradan Django nesneleri **değildir**. Bunun yerine, bunlar oluşturulmuş ancak kaydedilmemiş bir nesneyi ve ilgili tüm ilişki verilerini içeren özel “**DeserializedObject**(Geri Dönüştürülmüş Veriler)”örnekleridir.

DeserializedObject.save() yöntemi çağrıldığında nesne veritabanına kaydedilir.



Ayrıca btk.

Seri hale getirilmiş verilerdeki pk özniteliği mevcut değilse veya null ise, veritabanına yeni bir örnek kaydedilir.

Bu, serileştirilmiş temsilinezdaki veriler veritabanında bulunanlarla eşleşmese bile, “**deserialize**(Veri Serileştirmesinin kaldırılması)” işleminin verileri bozmayan bir işlem olmasını sağlar. Genellikle, bu “**DeserializedObject**(Geri Dönüştürülmüş Veriler)” örnekleriyle çalışma şu şekilde gerçekleşir:

```
for deserialized_object in serializers.deserialize("json", data):
    if object_should_be_saved(deserialized_object):
        deserialized_object.save()
```

Başka bir deyişle, genel kullanım şekli, nesneleri kaydetmeden önce serileştirilme işleminin kaldırılmış halini inceleyerek b unların kaydetmeye “uygun” olduğundan emin olmaktır. Veri kaynağınıza güveniyorsanız, bunun yerine nesneyi doğrudan kaydedip devam edebilirsiniz.

Django nesnesi, **deserialized_object.object** olarak incelenebilir. Serileştirilmiş verilerdeki alanlar bir modelde mevcut değilse, **ignorenonexistent** argümanı **True** olarak geçilmedikçe bir **DeserializationError** hatası verilir:

```
serializers.deserialize("json", data, ignorenonexistent=True)
```

Serileştirme Biçimleri ¶

Django, çeşitli serileştirme biçimlerini destekler; bunlardan bazıları için üçüncü taraf Python modüllerini yüklemeniz gerekir:

Tanımlayıcı	Bilgi
Xml	Basit bir XML lehçesine (formatına) dönüştürülür ve bu formattan geri dönüştürülür."
Json	json formatna paketlenir ve bu paketten geri açılır.
Jsonl	jsonl formatına paketlenir ve bu paketten geri açılır.
Yaml	YAML (YAML Ain't a Markup Language) formatına dönüştürür. Bu dönüştürücü, yalnızca PYYAML yüklü olduğunda kullanılabilir.